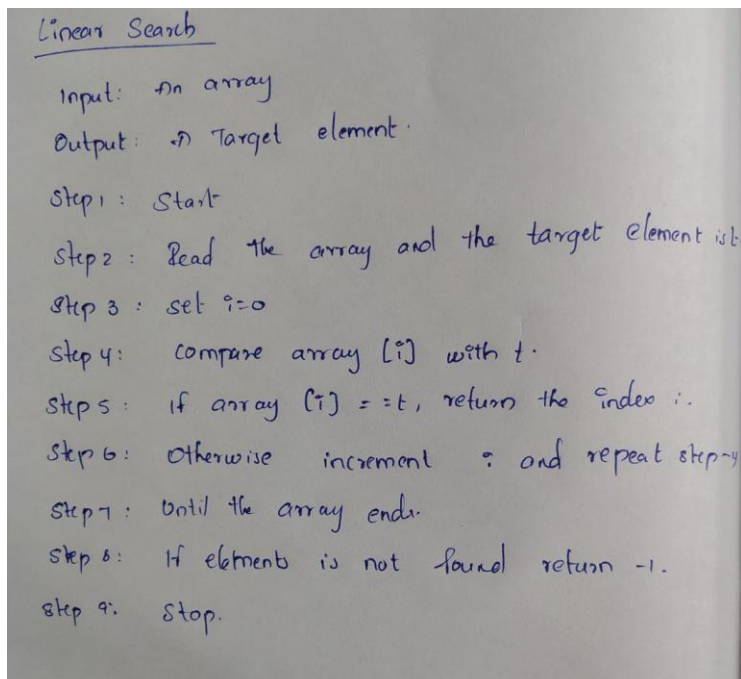


PRACTICAL: 02

Aim: To Implement Linear search and binary search and Time analysis their algorithms.

LINEAR SEARCH

Algorithm:



Program:

```
#include <iostream> using
namespace std;

int linearSearch(int arr[], int n, int key) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == key)
            return i;
    }
    return -1;
}

int main() {
    int arr[] = {10, 20, 30, 40, 50};
    int n = 5;    int key = 30;

    int result = linearSearch(arr, n, key);

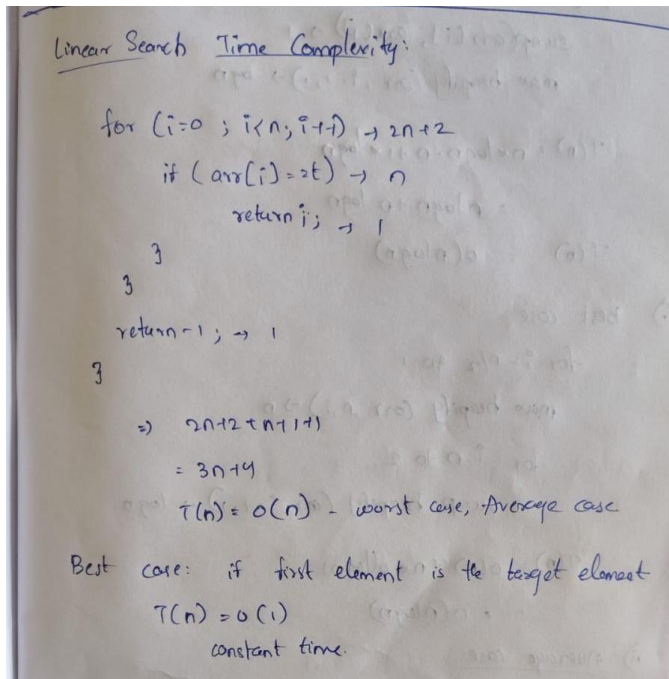
    if (result != -1)
        cout << "Element found at index " << result;
    else
        cout << "Element not found";

    return 0;
}
```

Output:

```
● PS C:\Users\pusap\Downloads\c++> & "C:\Users\pusap\Downloads\c++\LINEAR_SEARCH.exe"
  Element found at index 2
○ PS C:\Users\pusap\Downloads\c++>
```

Time Complexity:

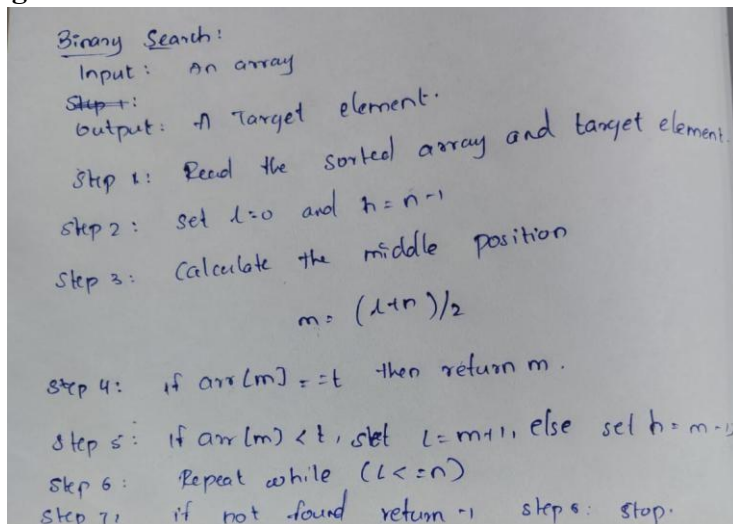


Conclusion:

Linear Search is a simple searching technique that checks each element one by one until the target element is found. It can be used on both sorted and unsorted arrays, but its time complexity is $O(n)$.

BINARY SEARCH

Algorithm:



Binary Search:
Input: An array
Step 1: Output: A Target element.
Step 1: Read the sorted array and target element.
Step 2: Set $l=0$ and $h=n-1$
Step 3: Calculate the middle position
 $m = (l+h)/2$
Step 4: If $arr[m] == t$ then return m .
Step 5: If $arr[m] < t$, set $l = m+1$, else set $h = m-1$.
Step 6: Repeat while $(l \leq n)$
Step 7: If not found return -1. Step 8: Stop.

Program: #include

<iostream> using

namespace std;

```
int binarySearch(int arr[], int n, int key) {
```

```
int low = 0, high = n - 1;
```

```
while (low <= high) {
```

```
int mid = (low + high) / 2;
```

```
if (arr[mid] == key)
```

```
return mid;
```

```
else if (arr[mid] < key)
```

```
low = mid + 1;
```

```
else
```

```
high = mid - 1;
```

```
}  
  
return -1;  
}  
  
int main() {  
    int arr[] = {10, 20, 30, 40, 50};  
    int n = 5;    int key = 40;  
  
    int result = binarySearch(arr, n, key);  
  
    if (result != -1)  
        cout << "Element found at index " << result;  
    else  
        cout << "Element not found";  
  
    return 0;  
}
```

Output:



```
PS C:\Users\pusap\Downloads\c++> C:\Users\pusap\Downloads\c++\Binarysearch.exe  
Element found at index 3  
PS C:\Users\pusap\Downloads\c++>
```

Time complexity:

Binary Search:

Worst case: if the element present at the end

$$T(n) = T(n/2) + 1$$
$$= T(n/2) + 2 \dots T(n/2^k) + k$$
$$\frac{n}{2^k} = 1 \text{ apply log on both sides.}$$
$$\log n = \log 2^k \quad T(n) = O(\log n)$$
$$k = \log_2 n$$

Best case:
if the target element is mid.

$$T(n) = O(1)$$

Average case:
random target.

$$T(n) = O(\log n)$$

Conclusion:

Binary Search is a faster searching technique that repeatedly divides a sorted array into two halves to find the target element. Its time complexity is $O(\log n)$, making it more efficient than Linear Search for large sorted arrays.